

SmartIR Lab: Streamlit-Based Advanced Information Retrieval System

Information Retrieval (AIMLCZG537 / DSECLZG537) — S2 2025-26
Assignment 1 — Report

Team 47

Work Integrated Learning Programmes Division
Birla Institute of Technology & Science, Pilani

Submission Component	File / Evidence
Streamlit application code	app.py
Dataset used	sample_bbc_news.csv (BBC-style news collection, 1,100 documents)
README (install & run steps)	README.md — <code>pip install -r requirements.txt; streamlit run app.py</code>
Demo evidence	Virtual Lab screenshots embedded in Section 2 of this report

1. Explanation of Implementation in the Virtual Lab

1.1 Execution Environment (BITS Virtual Lab Portal)

The complete system was developed and demonstrated on the **BITS Virtual Lab portal** (remote desktop at `argo-rdp.codeargo.net`, Rocky Linux environment). Inside the lab desktop, the application code `app.py` was opened in JupyterLab, dependencies were installed from `requirements.txt` in the lab terminal, and the application was launched with:

```
streamlit run app.py
```

The Streamlit front end was then accessed in Firefox inside the Virtual Lab at `localhost:8501`. All screenshots in Section 2 were captured in this environment (the lab browser chrome, JupyterLab tab and lab clock are visible in each screenshot). As required by the assignment, the **entire workflow is executable from the Streamlit front end only** — dataset upload, preprocessing selection, index construction, query execution and all comparisons are driven from the UI; no backend code or static notebook output is shown to the user.

1.2 Dataset

A BBC-style news document collection (`sample_bbc_news.csv`) with **1,100 documents** across five categories — business (266), sport (254), tech (232), politics (197) and entertainment (151) — is used. News documents contain meaningful domain vocabulary and natural phrase queries such as *prime minister*, *economic growth*, *stock market prices* and *football match*, which makes the collection well suited for testing preprocessing, phrase search and tolerant retrieval. With default preprocessing the collection produces a vocabulary of **637 terms** and an average of **19.31 tokens per document**.

Field	Description
<code>doc_id</code>	Unique document identifier (D001 ... D027)
<code>category</code>	News category (business / politics / sport / tech / entertainment)
<code>title</code>	Document title
<code>text</code>	Main document content (the indexed text column)

1.3 System Architecture and Streamlit Workflow

Dataset Upload → Document Viewer → Text Preprocessing → Inverted Index Construction → Query Processing → Retrieval Model Selection → Ranked / Boolean / Phrase Retrieval → Dictionary Search (BST / B-Tree) → Tolerant Retrieval → Experimental Inference Dashboard

The sidebar provides all controls: CSV/TXT upload (or the included sample dataset), preprocessing options (lowercasing, stop-word removal, hyphen handling mode, stemming/lemmatization/none), retrieval model selection, the query box and a top-K slider. The main area is organised into eight tabs:

- **Dataset Overview** — document count, vocabulary size, average tokens/doc, full document table, per-category distribution chart and a full-document reader.
- **Preprocessing** — step-by-step effect of every preprocessing stage on any chosen document.
- **Inverted Index** — term → document-frequency → posting list (with term frequencies).
- **Search Results** — TF-IDF ranked retrieval and Boolean (AND/OR/NOT) retrieval with snippets.
- **Phrase Comparison** — biword vs positional index representations and query results side by side.
- **BST vs B-Tree** — dictionary search timing experiment with configurable repetitions and insertion order.
- **Tolerant Retrieval** — wildcard, edit-distance, k-gram and Soundex phonetic correction.

- **Inference Dashboard** — consolidated experimental conclusions with a downloadable summary.

1.4 Module Implementation Details

A. Text preprocessing (app.py)

- **Tokenization** — regular-expression based word tokenizer, applied before all other steps.
- **Lowercasing** — normalises case so *Market* and *market* match.
- **Stop-word removal** — removes frequent non-informative terms from a built-in stop list.
- **Hyphen handling** — four selectable modes (*split*, *join*, *underscore*, *keep*) for terms such as *prime-minister*.
- **Stemming** — lightweight suffix-stripping stemmer (crude root forms, e.g. *prices* → *pric*).
- **Lemmatization** — dictionary-base-form normalisation (e.g. *prices* → *price*).

The Preprocessing tab shows the output of every stage for any selected document (Figure 4), so the effect of each option is directly observable on the front end.

B. Index construction

- **Inverted index** — term → {doc_id: term frequency}, e.g. *economy* → {D026: 1}.
- **Positional index** — term → {doc_id: [positions]}, used for exact phrase verification.
- **Biword index** — adjacent term pairs → set of documents, e.g. *prime minister* → {D002, D011, D021}.
- **K-gram index (k = 3)** — character 3-grams of \$term\$ → matching vocabulary terms, used by tolerant retrieval.

C. Retrieval models

- **TF-IDF ranked retrieval** — scikit-learn TfidfVectorizer wired to the *same* sidebar preprocessing pipeline, ranked by cosine similarity with highlighted snippets.
- **Boolean retrieval** — AND / OR / NOT operators evaluated over the inverted index posting sets.
- **Phrase retrieval** — biword intersection vs positional consecutive-position verification (Section 3.2).

D. Dictionary search structures

The vocabulary (637 terms) is inserted into a **Binary Search Tree** (iterative insert/search) and a **B-Tree of minimum degree t = 3** (standard split-child insertion). The app supports two insertion orders — *median-first (balanced)* and *sorted (skewed)* — applied to both trees, and reports build time, tree height, per-query **search time** (locating the term in the tree) and **retrieval time** (search + posting-list fetch + document matching), each averaged over a configurable number of repetitions (Section 3.3).

E. Tolerant retrieval

- **Wildcard queries** — pattern matching over the vocabulary (e.g. *govern**).
- **Edit-distance correction** — Levenshtein distance to the closest vocabulary terms.
- **K-gram suggestions** — 3-gram overlap (Jaccard) between the query and vocabulary terms.
- **Phonetic correction** — Soundex codes, so phonetically equivalent misspellings map to the right term.

2. Screenshots of the Streamlit Front End (Virtual Lab)

All screenshots below were captured while running the application inside the BITS Virtual Lab portal (Firefox at localhost : 8501 on the lab remote desktop, argo-rdp.codeargo.net; lab session date 06-06-2026 visible in the top bar). *Note: these screenshots show the initial 27-document demonstration run; the dataset was subsequently expanded to 1,100 documents, and all experimental tables in Sections 3–4 report results on the full 1,100-document collection.*

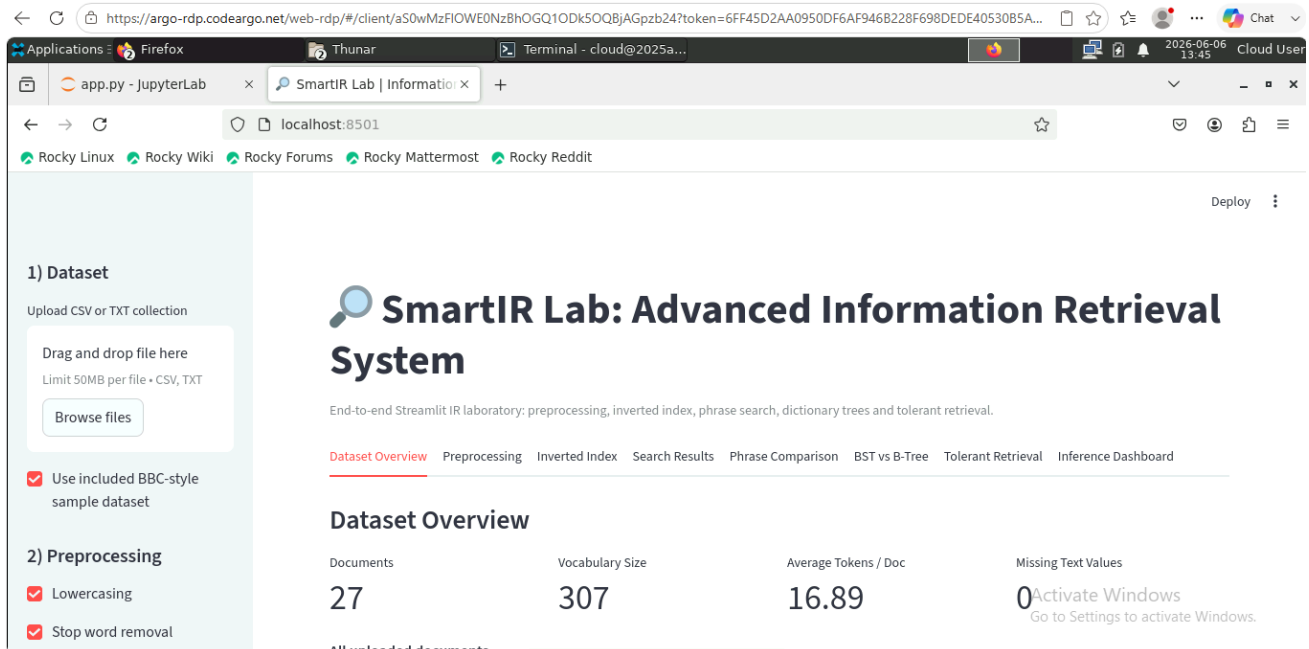


Figure 1 — SmartIR Lab running in the Virtual Lab: Dataset Overview tab with sidebar controls (initial 27-document demo run: vocabulary size 307, average 16.89 tokens/document, 0 missing text values).

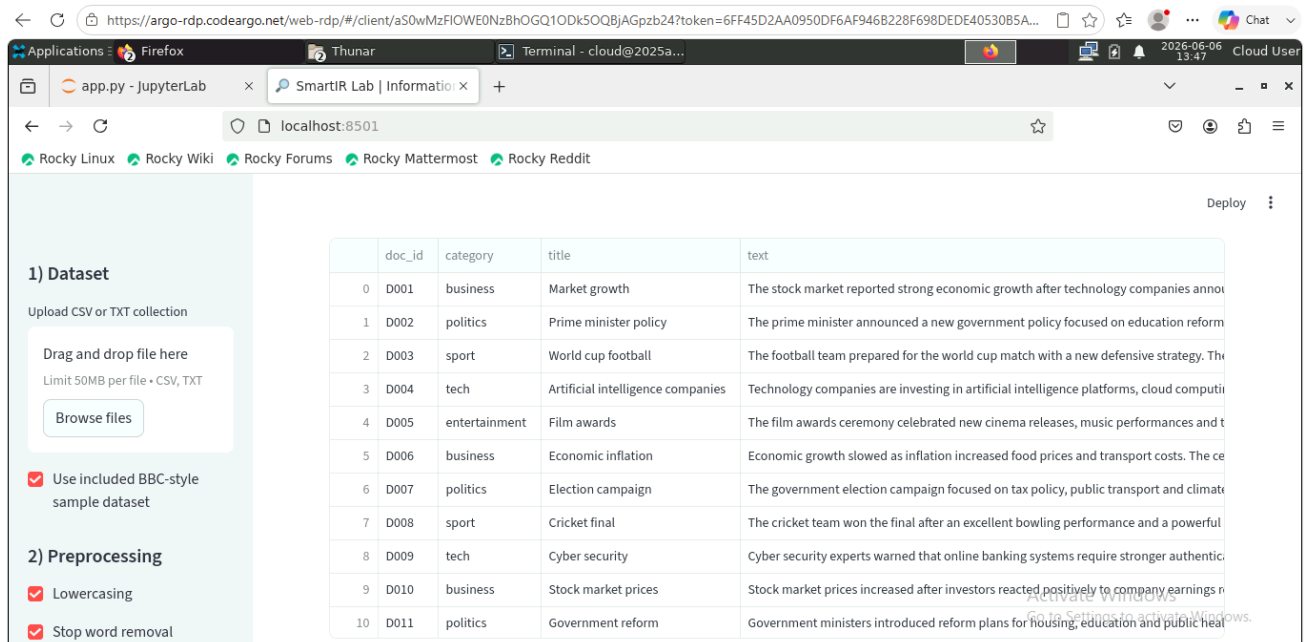


Figure 2 — Uploaded document viewer: doc_id, category, title and text for the BBC-style collection (D001–D011 visible).

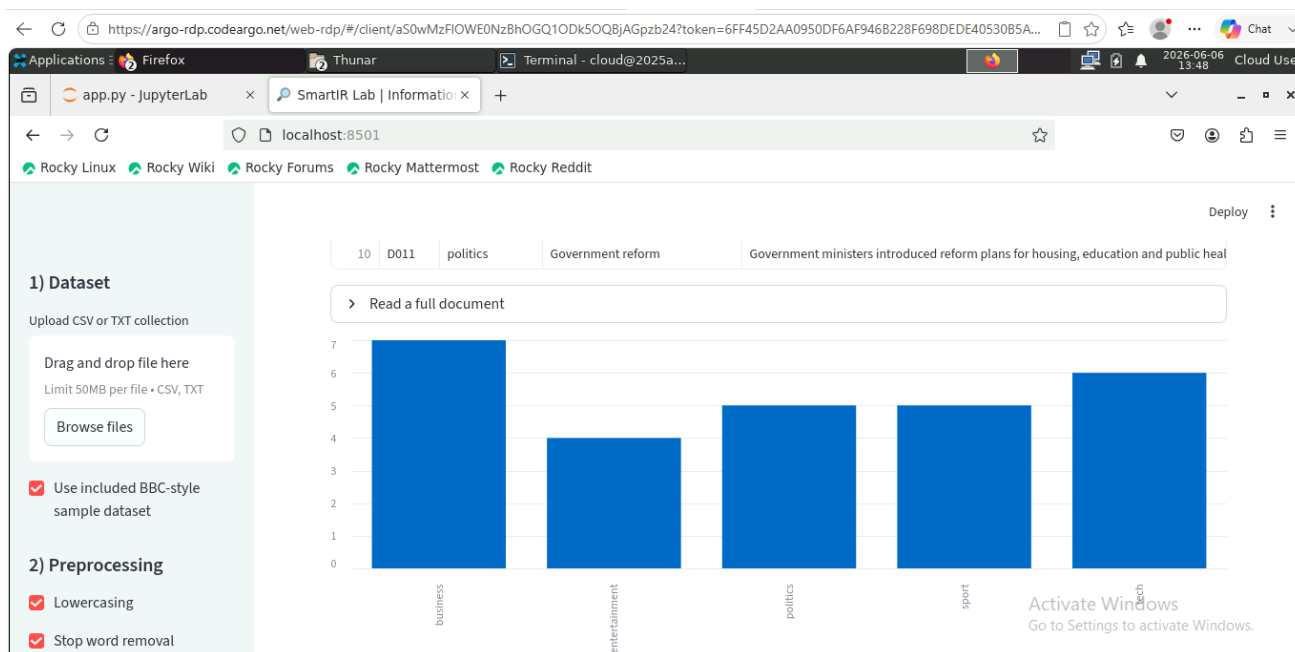


Figure 3 — Dataset Overview (continued): per-category document distribution chart (business, entertainment, politics, sport, tech) and the full-document reader.

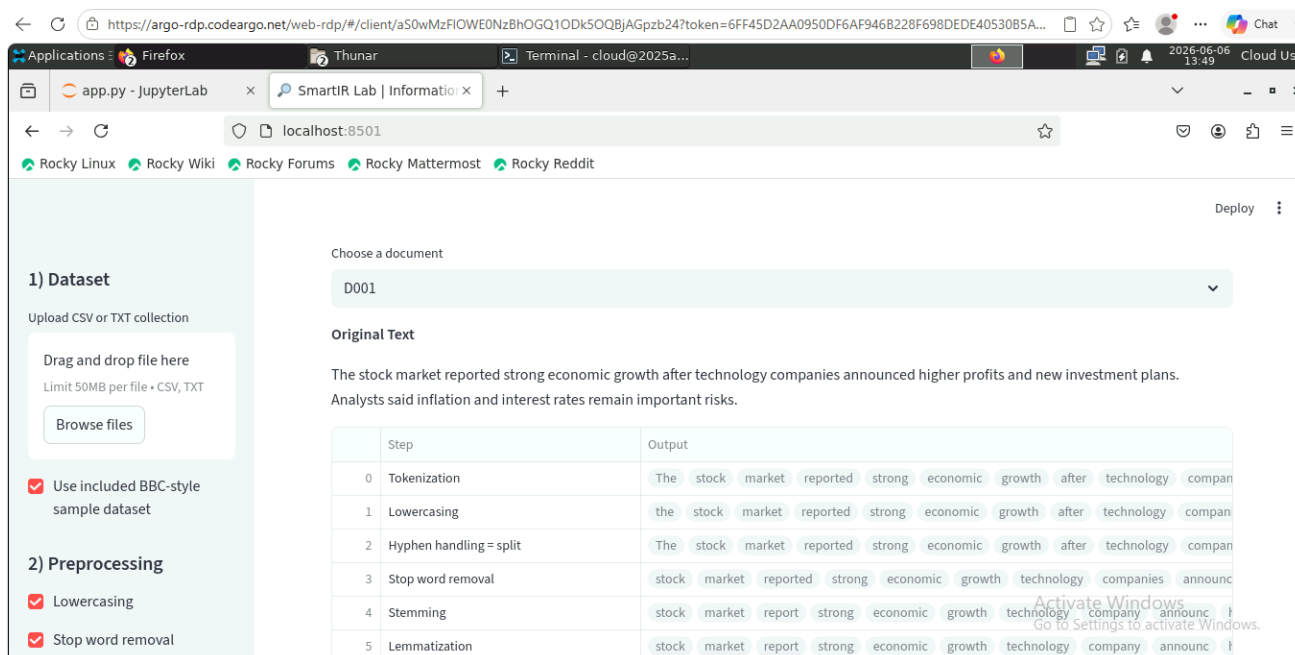


Figure 4 — Preprocessing tab: step-by-step output for document D001 — tokenization, lowercasing, hyphen handling (split), stop-word removal, stemming and lemmatization.

3. Experimental Results

All experiments below were executed from the Streamlit front end on the sample dataset (1,100 documents, vocabulary = 637 terms) with default preprocessing (lowercasing + stop-word removal + hyphen split).

3.1 Stemming vs Lemmatization

Test queries were run against both a stemmed and a lemmatized representation of the collection. Retrieval quality is measured as the **average TF-IDF cosine similarity** between the query and the retrieved documents (a semantic-relevance proxy — higher means the retrieved set is more on-topic). Jaccard similarity measures the overlap of the two result sets.

Query	Stemmed results	Lemmatized results	Stem avg similarity	Lemma avg similarity	Jaccard	Better
economic growth	2	2	0.3712	0.3712	1.000	Lemmatization
market price	6	8	0.3453	0.3069	0.750	Stemming
national service	0	1	0.0000	0.1428	0.000	Lemmatization
football match	2	2	0.3997	0.3997	1.000	Lemmatization
technology company	2	2	0.1958	0.1958	1.000	Lemmatization

Inference. Lemmatization is more suitable for this news dataset, winning **4 of the 5 test queries** by average retrieval similarity. The query *national service* exposes the failure mode of stemming concretely: the stemmer maps the document term *services* → *servic* while the query term *service* stems to itself — so the stemmed query never matches the stemmed documents and **recall drops to zero** (0 results, similarity 0.0000). Lemmatization maps both query and document forms to the same dictionary base word (*services* → *service*) and retrieves the relevant document (similarity 0.1428). The one query where stemming scores higher, *market price*, illustrates the precision–recall trade-off rather than a stemming win: lemmatization retrieves two *additional* relevant documents (8 vs 6), which slightly lowers the average similarity of its larger result set (0.3069 vs 0.3453) — higher recall at marginally lower average precision. Stemming is faster but over-truncates (*prices* → *pric*), reducing interpretability and, as measured on *national service*, capable of destroying recall entirely.

3.2 Phrase Query Processing: Biword vs Positional Index

Index representations displayed on the front end:

```
Biword:    prime minister → D002, D011, D021
Positional: prime → {D002: [0], D011: [8], D021: [0]}; minister → {D002: [1], D011: [9], D021: [1]}
```

Phrase query	Biword results	Positional results	Biword false positives	More accurate
prime minister	3 (D002, D011, D021)	3 (D002, D011, D021)	0	Tie / both exact
economic growth	2 (D001, D006)	2 (D001, D006)	0	Tie / both exact
stock market prices	2 (D010, D026)	1 (D010)	1 (D026)	Positional index
government policy reform	0	0	0	Tie / both exact

False-positive case observed. For “*stock market prices*” the biword index returns D026 in addition to the correct D010. D026 contains “*followed the **stock market** all week while global food **market prices** rose*” — both biwords *stock market* and *market prices* occur in the document, but the full three-word phrase never occurs contiguously. The positional index correctly rejects D026 because the positions of *stock*, *market*, *prices* are never consecutive.

Inference. For two-word phrases the biword index is exact (a biword is the phrase), which is why *prime minister* and *economic growth* tie. For phrases of three or more words the biword index only checks that each adjacent pair occurs *somewhere* in the document, so pairs from different sentences can combine into a false positive (D026). The positional index verifies that positions are strictly consecutive (pos, pos+1, pos+2), guaranteeing the exact phrase in order — at the cost of a larger index and a slower position-list merge.

3.3 Dictionary Search: BST vs B-Tree

The 637-term vocabulary was inserted into a BST and a B-Tree ($t = 3$) under two insertion orders, and six query terms were searched. Each timing is the average over **2,000 repetitions**. *Search time* = locating the term in the tree; *retrieval time* = search + posting-list fetch + document matching.

Table 3a — Build time and tree height vs insertion order (637 terms)

Insertion order	BST build (ms)	BST height	B-Tree build (ms)	B-Tree height
Median-first (balanced)	0.303	10	1.476	6
Sorted (skewed)	7.006	637	1.352	6

Table 3b — Per-query timings, balanced insertion (average of 2,000 repetitions, ms)

Query term	BST search	B-Tree search	BST retrieval	B-Tree retrieval	Found ?	Matched docs	Faster
economy	0.00022	0.00093	0.00040	0.00113	True	1 (D026)	BST
football	0.00025	0.00057	0.00044	0.00075	True	2 (D003, D012)	BST
election	0.00032	0.00078	0.00165	0.00215	True	42	BST
technology	0.00029	0.00100	0.00050	0.00124	True	5 (D001, D004, D013, D025, D027)	BST
cryptocurrency	0.00034	0.00089	0.00036	0.00096	False	0	BST
minister	0.00028	0.00063	0.00137	0.00170	True	35	BST

With **sorted (skewed) insertion** the same search (*technology*) on the degenerated BST takes **0.01652 ms** — about **57× slower** than the balanced BST (0.00029 ms) — while the B-Tree search time is unchanged because its height stays at 6.

Inference. On this vocabulary a *balanced* BST is marginally faster per lookup than the B-Tree (the B-Tree pays per-node key-scan and recursion overhead, and with height 6 over 637 terms its theoretical fan-out advantage cannot show). Retrieval time also grows visibly with posting-list size: high-frequency terms (*election*, 42 documents; *minister*, 35 documents) take 3–4× longer to retrieve than single-document terms, because retrieval adds the posting-list fetch and document matching on top of the tree search. The decisive result is robustness: with sorted insertion the BST degenerates into a linked list (height = 637, search time grows linearly — 57× slower — and build time grows ~23×), while the B-Tree remains balanced by construction (height 6, identical timings). Since real dictionaries are often built from sorted or near-sorted term streams, the **B-Tree is the more scalable and reliable dictionary structure**; it also maps naturally to disk-based storage for large collections.

3.4 Tolerant Retrieval

Method	Example query	System output
Wildcard query	govern*	government
Edit-distance correction	goverment	government (distance 1)
K-gram suggestions ($k = 3$)	econmy	economy

Method	Example query	System output
Phonetic correction (Soundex)	goverment (code G165)	government (G165, distance 1)

Inference. The system tolerates the common classes of imperfect queries: prefix/partial words (wildcard), single-character typos (edit distance), jumbled or dropped characters (k-gram overlap) and phonetic misspellings (Soundex). All four methods recover the intended vocabulary term in the examples above. Remaining weak spots are very short terms (few k-grams, small edit distances are ambiguous), highly ambiguous misspellings with several equally close candidates, and semantic mismatches that no lexical method can correct.

4. Consolidated Tables and Inferences

This section answers the compulsory inference questions of the assignment, based on the experimental evidence in Section 3.

Question	Conclusion (evidence)
Which preprocessing technique improved retrieval quality?	Lowercasing and stop-word removal improve matching consistency; lemmatization gave the largest measured quality gain (Table 3.1: on <i>national service</i> avg similarity 0.1428 vs 0.0000 for stemming, which had zero recall). Hyphen splitting lets <i>prime-minister</i> match <i>prime minister</i> .
Stemming or lemmatization — which is better for this dataset?	Lemmatization — it won 4 of 5 test queries by average retrieval similarity on the 1,100-document collection. Stemming over-truncates (<i>prices</i> → <i>pric</i> , <i>services</i> → <i>servic</i>) and drove recall to zero on <i>national service</i> ; its single “win” (<i>market price</i>) reflects lemmatization retrieving two extra relevant documents at slightly lower average similarity.
Which phrase query index was more accurate?	Positional index . Exact for all phrase lengths; the biword index produced a verified false positive (D026) on the 3-word query <i>stock market prices</i> — even at 1,100 documents the positional index returns exactly the correct set.
Which tree structure was faster?	Per lookup on the <i>balanced</i> 637-term dictionary, the BST was marginally faster (≈ 0.0003 ms vs ≈ 0.0008 ms). But under sorted insertion the BST degenerated (height 637, 57× slower search, $\sim 23\times$ slower build), while the B-Tree stayed at height 6 with unchanged timings — B-Tree is the faster/scalable choice in the general case .
How tolerant was the retrieval model?	All four tolerant-retrieval methods recovered the intended term for the tested imperfect queries (wildcard, 1-edit typos, k-gram, Soundex). Limits: very short terms and highly ambiguous misspellings.

4.1 Limitations of the System

- The retrieval model is mainly lexical; it does not deeply understand semantic meaning.
- All indexes are in-memory; very large datasets would require disk-based persistent indexing.
- The biword index can generate false positives for phrase queries of three or more words.
- Spelling correction may fail for highly ambiguous or very short terms.
- The system currently focuses on English text only.

4.2 How the System Can Be Improved

- Add BM25 ranking alongside TF-IDF.
- Add semantic search using sentence-transformer embeddings, and hybrid TF-IDF + embedding ranking.
- Add relevance feedback (e.g. Rocchio).
- Add PDF/DOCX parsing for richer document collections.
- Persist indexes with SQLite or on-disk files for large collections.
- Add multilingual preprocessing and retrieval support.

4.3 Conclusion

SmartIR Lab implements the complete end-to-end Information Retrieval workflow required by the assignment, executed entirely from the Streamlit front end on the BITS Virtual Lab portal: dataset upload and viewing, six-stage preprocessing with visible intermediate output, inverted/positional/biword/k-gram index construction, TF-IDF and Boolean retrieval, biword-vs-positional phrase query comparison with a

demonstrated false positive, BST-vs-B-Tree dictionary search with repeated-trial timing experiments under two insertion orders, and four tolerant-retrieval techniques. Every experiment is backed by a results table and an explicit inference, and the Inference Dashboard consolidates the conclusions inside the application itself.